

Transfer and Shared Learning for Atari Control

Santiago Restrepo

contact@waajacu.com <https://waajacu.com/>

Shared perception, specialized control, and measurable retention

Abstract—This approach learns Atari control from pixels through policy optimization, feature transfer, and balanced practice across games. A shared visual encoder supports specialized policies. The design separates perception from control, isolates the effect of transferred knowledge, and checks retention explicitly. Native parallel simulation, independent playback, and preserved experiment evidence make the learning process efficient and inspectable.

Index Terms—Atari, reinforcement learning, transfer learning, shared representation.

I. LEARN CONTROL FROM VISUAL EXPERIENCE

The Arcade Learning Environment (ALE) supplies screens, actions, and game rewards [1]. Grayscale conversion, resizing, and temporal pooling produce compact observations. Stacking recent screens exposes motion to a convolutional encoder without object coordinates or handwritten gameplay rules. A registry records each game’s ROM identity, action count, and evaluation criterion.

The shared encoder feeds a game-specific actor and critic:

$$z_t = f_{\theta}(o_t), \quad a_t \sim \pi_{\phi_g}(\cdot | z_t), \quad v_t = V_{\psi_g}(z_t). \quad (1)$$

Here o_t is the screen stack, g selects the game’s heads, and v_t estimates future discounted training reward. Shared visual features remain separate from game-specific action meanings.

II. IMPROVE POLICIES WITH GAME REWARDS

Proximal Policy Optimization (PPO) alternates on-policy interaction with minibatch learning [2]. Actions are sampled from the actor. Generalized advantage estimation uses rewards and critic predictions to identify better-than-expected actions. A clipped policy objective discourages excessive changes, critic regression improves prediction, and an entropy incentive encourages exploration.

Rewards accumulate over repeated actions and are clipped for learning; evaluation retains raw returns. Success thresholds are evaluation criteria, not reward bonuses. Life loss preserves episode continuity. Time-limit truncation bootstraps from the critic rather than being treated as a natural terminal outcome.

III. ISOLATE THE EFFECT OF FEATURE TRANSFER

Initialize a target encoder from a learned source policy. Create fresh target actor and critic heads and a fresh optimizer, then fine-tune the complete network. This reuses perception without importing incompatible actions or source optimizer history.

Compare against learning from scratch with matched target interaction budgets, seeds, initial heads, and evaluation conditions. Account for source pretraining separately. This isolates

whether visual reuse helps adaptation instead of assuming that transferred weights help.

IV. BALANCED SHARED LEARNING AND RETENTION

To add a game, restore the shared encoder and existing actor–critic heads, initialize new heads, and start a fresh joint optimizer. Collect fresh experience from earlier games alongside the new game, keeping their objectives active throughout adaptation.

Balance environments and minibatches across games, normalize advantages within each game, and weight game losses equally. This limits domination by interaction volume or advantage scale while allowing every task to update shared features. Compare each policy before and after adaptation: continued practice supports retention, but conflicting gradients can still cause forgetting.

V. EFFICIENT AND INSPECTABLE EXECUTION

Native C++ workers advance emulators on the CPU; batched inference and optimization use the GPU through LibTorch. Independent browser playback loads saved policies without pacing training. A reproducible Debian environment separates dependency installation from lifecycle and experiment commands.

Preserve joint checkpoints, game-specific inference exports, settings, and source lineage. Atomic checkpoint publication lets concurrent readers access complete artifacts without coupling playback to the learner’s mutable state.

VI. MAKE EVALUATION A DESIGN CONTRACT

Freeze policies and use consistent preprocessing, sticky actions, reset rules, and full-game termination. Evaluation selects the most probable action. Pair seeds for retention comparisons and reserve separate seeds for final confirmation. Distinguish natural endings from capped episodes; assess each game’s criterion independently. Preserve hashes and episode traces so each claim identifies its policy, training lineage, and evaluation protocol.

REFERENCES

- [1] M. C. Machado *et al.*, “Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents,” *Journal of Artificial Intelligence Research*, vol. 61, pp. 523–562, 2018, arXiv:1709.06009.
- [2] J. Schulman *et al.*, “Proximal Policy Optimization Algorithms,” *arXiv preprint*, 2017, arXiv:1707.06347.